

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5112

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

Total Marks: 50

Annexure A

Code of Conduct:

I G. Srivalli (192365014) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G. Srivalli

Annexure A

1. Secure Password Storage in Web Applications

A web application stores user passwords and needs to protect them against database breaches.

- A. Explain why storing plaintext passwords is insecure.
 - B. Suggest a secure method for storing passwords and justify your choice.
-

2. Digital Signature for Document Authentication

A company wants to ensure that official documents sent electronically are authentic and unaltered.

- A. Explain how digital signatures can be used in this scenario.
 - B. Justify how this method ensures both integrity and non-repudiation.
-

3. Preventing Replay Attacks in Online Transactions

An online payment system is vulnerable to replay attacks where attackers resend previously captured transactions.

- A. Explain how replay attacks occur.
 - B. Suggest a cryptographic technique to prevent such attacks and justify your answer.
-

4. Secure Communication over Public Wi-Fi

A user accesses sensitive information using a public Wi-Fi network without encryption.

- A. Identify the risks involved in this scenario.
 - B. Recommend a secure communication method and justify how it protects user data.
-

5. Data Integrity in Cloud Storage

An organization stores critical data in a cloud server and wants to ensure that the data is not altered.

- A. Explain how hashing can be used to verify data integrity.
- B. Suggest an additional mechanism to enhance trust and justify your answer.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1: Secure Password Storage in Web Applications

1.A. Why Storing Plaintext Passwords is Insecure

Overview: Storing user passwords in plaintext format within a database represents one of the most severe security anti-patterns in modern web software engineering. When passwords are saved in raw readable form, any security compromise of the underlying persistence layer directly exposes all user credentials. The primary risks and technical vulnerabilities include:

- **Exposure to Database Breaches:** A single SQL Injection (SQLi), remote code execution, server misconfiguration, or unauthorized database backup exposure allows attackers to instantly dump the user credential table. Plaintext storage provides zero defense-in-depth, rendering every account vulnerable immediately upon read access.
- **Insider Threats & Excessive Privilege Exposure:** Database administrators, system engineers, third-party contractors, or support personnel with database access can view user credentials without authorization, leading to insider threat exploitation and credential harvesting.
- **Credential Stuffing & Cross-Site Vulnerability:** Over 60-70% of Internet users reuse the same password across multiple online services (banking, email, e-commerce, enterprise portals). A plaintext breach on one web application immediately compromises the user's digital identity across the entire Internet through automated credential stuffing attacks.
- **Non-Compliance & Regulatory Liabilities:** Regulatory bodies under standards such as PCI-DSS (Requirement 8.3), HIPAA, NIST SP 800-63B, and EU GDPR mandate robust encryption or cryptographic hashing for authentication data. Plaintext password storage results in severe legal liabilities, financial penalties, and catastrophic reputational damage.

1.B. Recommended Secure Method & Technical Justification

Recommendation: The industry standard and mathematically robust solution for password protection is Salted Adaptive Password Hashing utilizing a modern Memory-Hard Key Derivation Function (KDF) such as Argon2id, bcrypt, or PBKDF2-HMAC-SHA256.

Technical Implementation Blueprint: Argon2id / bcrypt

1. Unique Salt Generation: For every password registration or update, generate a cryptographically secure pseudo-random 128-bit (16-byte) Salt using CSPRNG.
2. Adaptive Key Derivation: Pass the plaintext password combined with the unique salt into the Argon2id (or bcrypt) algorithm with calibrated work factors (Time Cost $t=3$, Memory Cost $m=64\text{MB}$, Parallelism $p=4$).
3. Storage Format: Store the resulting hash alongside the salt and algorithm parameters in the database format: `$argon2id$v=19$m=65536,t=3,p=4$<salt>$<hash>`.
4. Verification: During login, retrieve the stored salt and parameters, re-hash the incoming candidate

password, and perform a Constant-Time String Comparison to eliminate timing side-channel attacks.

- **Defeat of Rainbow Table Attacks:** Precomputed lookup tables (Rainbow Tables) rely on matching static hashes. Adding a unique 128-bit salt guarantees that identical passwords yield completely distinct hash strings, invalidating all precomputed rainbow tables.
- **Work Factor Adaptability & Resistance to GPU/ASIC Hardware:** Algorithms like Argon2id and bcrypt feature adjustable work factors (cost parameters). As hardware processing power increases over time, administrators can increase the memory and CPU work factor to keep offline brute-forcing computationally prohibitive without changing the underlying codebase.
- **Memory-Hard Defense against Hardware Acceleration:** Standard SHA-256 or MD5 hashes can be evaluated billions of times per second on modern GPUs. Argon2id incorporates memory hardness (requiring heavy RAM access patterns), which severely throttles hardware-accelerated parallel cracking attempts.

Question 2: Digital Signature for Document Authentication

2.A. How Digital Signatures Work in Document Authentication

Mechanism Overview: A Digital Signature is a mathematical mechanism based on Asymmetric (Public Key) Cryptography that binds an identity to a specific digital document or message. It operates through a combination of cryptographic hash functions (e.g., SHA-256) and asymmetric key pairs (RSA or ECDSA).

Digital Signature Workflow Process

Step 1: Document Digest Generation (Sender/Signer)

- The sender computes a cryptographic hash digest $H = \text{SHA-256}(\text{Document})$. This digest is a fixed-length (256-bit) unique mathematical fingerprint.

Step 2: Signature Creation

- The sender encrypts/signs the hash digest H using their Private Key ($K_{\text{private_sender}}$) to produce the Signature: $S = \text{Sign}(H, K_{\text{private_sender}})$.
- The document and signature bundle $\{\text{Document}, S\}$ are transmitted to the recipient along with the sender's X.509 Digital Certificate.

Step 3: Verification Process (Recipient)

- The recipient decrypts signature S using sender's Public Key ($K_{\text{public_sender}}$) to extract original digest $H_{\text{original}} = \text{Decrypt}(S, K_{\text{public_sender}})$.
- The recipient independently computes a new hash digest $H_{\text{computed}} = \text{SHA-256}(\text{Document_received})$.
- If $H_{\text{original}} == H_{\text{computed}}$, the signature is mathematically VERIFIED.

2.B. Justification: Ensuring Integrity and Non-Repudiation

Security Guarantees: Digital signatures provide rigorous security guarantees through the mathematical properties of one-way hash functions and asymmetric cryptography:

- **Data Integrity Assurance:** Cryptographic hash functions satisfy the Strict Avalanche Criterion: changing even a single bit or character in the document completely alters the resulting hash value (H_{computed}). If an attacker alters the document during transit or storage, H_{computed} will fail to match H_{original} decrypted by the public key. This mathematical mismatch alerts the recipient instantly that tampering occurred.
- **Non-Repudiation Assurance:** Non-repudiation guarantees that the signer cannot deny having authored or authorized the document. Because $K_{\text{private_sender}}$ is strictly confidential to the legitimate owner, only the holder of the private key could have created a signature that successfully decrypts with $K_{\text{public_sender}}$. Furthermore, binding the public key to identity via a trusted Public Key Infrastructure (PKI) Certificate Authority (CA) provides legal enforceability under electronic signature laws.

Question 3: Preventing Replay Attacks in Online Transactions

3.A. How Replay Attacks Occur

Attack Mechanics: A Replay Attack is a network-level or protocol-level attack where an adversary eavesdrops on a secure network transmission, intercepts a valid data payload (e.g., a signed funds transfer request), and subsequently resends (replays) that exact payload to the recipient system to trick it into performing duplicate actions (such as authorizing duplicate wire transfers).

- **Financial Transaction Example:** In an unmitigated payment gateway, an attacker captures an encrypted transaction payload representing: 'Transfer \$1,000 from Account A to Account B (Signed by User A)'. Even if the attacker cannot decrypt or read the contents of the payload, retransmitting the exact intercepted byte stream causes the server to execute the transaction a second time, resulting in unauthorized financial loss.
- **Why Encryption Alone Fails:** Replay attacks bypass traditional confidentiality controls because the replayed data is legitimately signed and correctly encrypted originally. Standard cryptographic validation passes unless specific freshness controls are built into the payment protocol.

3.B. Cryptographic Techniques to Prevent Replay Attacks & Justification

Prevention Strategy: To counter replay attacks effectively, cryptographic protocol designers incorporate Session Freshness Mechanisms. Two primary, highly complementary techniques are employed:

Primary Countermeasures for Replay Attacks

Technique 1: Cryptographic Nonces (Number Used Once)

- Before initiating a transaction, the server issues a high-entropy random integer N (Nonce) to the

client.

- The client includes N inside the cryptographically signed transaction payload: $\text{Payload} = \text{Sign}(\text{Data} + \text{Nonce}, K_{\text{private}})$.
- The server maintains a database of recently used Nonces. Once processed, Nonce N is marked as spent. Replayed messages with used nonces are rejected immediately.

Technique 2: Synchronized Timestamps with Short Time-to-Live (TTL) & Sequence Numbers

- Each request includes UTC Timestamp T_{sent} and an incremental Sequence Number S_{seq} .
- Server validates that $|T_{\text{current}} - T_{\text{sent}}| < \Delta_{\text{threshold}}$ (e.g., 30 seconds) using Network Time Protocol (NTP) synchronization.
- Requests outside the valid window or with duplicate sequence numbers are dropped.

- **Technical Justification:** Nonces ensure absolute uniqueness per request. Even if an attacker captures payload $P(N_1)$, attempting to resend $P(N_1)$ fails because the server's state machine rejects any reused nonce value. Timestamps enforce temporal freshness, restricting the attack execution window to mere milliseconds.

Question 4: Secure Communication over Public Wi-Fi

4.A. Identified Risks on Unencrypted Public Wi-Fi

Threat Landscape: Public Wi-Fi networks (open IEEE 802.11 hotspots in cafes, airports, or hotels) operate as shared untrusted media. Connecting without proper end-to-end protection exposes users to critical threats:

- **Passive Eavesdropping & Packet Sniffing:** Attacker devices on the same Wi-Fi subnet can put their network interfaces into promiscuous mode to capture raw unencrypted TCP/UDP frames, reading session tokens, plain HTTP cookies, passwords, and sensitive emails.
- **Evil Twin Access Points & Man-in-the-Middle (MitM):** Attackers set up rogue access points broadcasting identical SSIDs (e.g., 'Airport_Free_WiFi'). User devices auto-connect, allowing the rogue AP to act as a Man-in-the-Middle, decrypting or modifying all routed traffic.
- **Address Resolution Protocol (ARP) Spoofing:** Exploiting weak ARP protocols on local subnets to associate the attacker's MAC address with the default gateway's IP address, forcing all local network traffic through the attacker's machine.
- **SSL/TLS Stripping & Session Hijacking:** Tools like SSLstrip downgrade user connections from secure HTTPS to plaintext HTTP, stripping transport-layer protection and harvesting authentication credentials.

4.B. Recommended Secure Communication Method & Justification

Recommendation: To fully neutralize public network vulnerabilities, a multi-layered security strategy combining Transport Layer Security (TLS 1.3 / HTTPS) and an Encrypted Virtual Private Network (VPN) using WireGuard or IPsec is recommended.

Defense Architecture: VPN + TLS 1.3

1. Network-Level Protection (Virtual Private Network):

- Encapsulates ALL device traffic into a secure AES-256-GCM encrypted tunnel using WireGuard or OpenVPN protocols from device to trusted VPN Gateway.
- Obfuscates DNS queries, preventing local network attackers or local Wi-Fi router admins from inspecting destination IP addresses or domain requests.

2. Application-Level Protection (TLS 1.3 / HTTPS + HSTS):

- Enforces strict end-to-end encryption between browser and application backend.
- HTTP Strict Transport Security (HSTS) prevents SSL stripping downgrade attacks.

- **Technical Justification:** Even if an attacker controls the Wi-Fi access point or executes ARP spoofing, all captured packet payloads consist of high-entropy cryptographic ciphertext. Without the VPN session keys or TLS private key, the captured traffic is completely unreadable and unmodifiable.

Question 5: Data Integrity in Cloud Storage

5.A. How Hashing Verifies Data Integrity

Integrity Verification Process: Cryptographic Hashing (e.g., SHA-256) is a foundational building block for verifying that stored cloud data remains unaltered over time or across storage migrations.

- **Hash Digest Baseline:** When a file F is uploaded to cloud storage, the owner computes hash $H_{\text{orig}} = \text{SHA256}(F)$ locally and stores H_{orig} in a secure location. To verify data integrity, the owner downloads or queries the stored file F' and computes $H_{\text{check}} = \text{SHA256}(F')$. If $H_{\text{orig}} == H_{\text{check}}$, the data is guaranteed to be untouched.
- **Mathematical Properties:** Hash functions exhibit Pre-image Resistance, Second Pre-image Resistance, and Collision Resistance. The Avalanche Effect ensures that modifying even a single byte in a 10 GB file generates a completely different 256-bit hash digest, rendering undetected data corruption or intentional tampering mathematically impossible.

5.B. Additional Mechanism to Enhance Trust & Justification

Enhanced Trust Mechanism: While basic hashing detects accidental silent corruption (bit rot), it does not protect against malicious cloud administrators who can alter both the stored data AND its associated stored hash digest. To establish absolute trust, an advanced cryptographic structure is required: Keyed-Hash Message Authentication Codes (HMAC) combined with Merkle Trees (Hash Trees) or Proof of Retrievability (PoR).

Advanced Trust Architecture: Merkle Trees & Proof of Retrievability (PoR)

1. Merkle Tree Block-Level Integrity:

- Large files are divided into data blocks (B_1, B_2, ..., B_n). Each block is hashed, and pairs of hashes are recursively hashed up to a single Merkle Root Hash.
- The client retains only the lightweight Merkle Root. The cloud server can prove any block's integrity using $O(\log N)$ hash path proofs without downloading the entire dataset.

2. Keyed-Hash Authentication (HMAC) / Digital Signatures:

- Hashes are authenticated using HMAC-SHA256 with a secret key known only to the data owner:
$$\text{HMAC} = \text{SHA256}(K \text{ XOR opad} || \text{SHA256}(K \text{ XOR ipad} || \text{Data}))$$
- The cloud provider cannot forge a valid HMAC after altering data because it lacks secret key K.

- **Technical Justification:** Basic hashing assumes a trustworthy storage environment. HMACs and Merkle Trees extend integrity into a Zero-Trust Model where the storage provider itself is treated as untrusted. Remote Data Auditing permits continuous integrity monitoring without bandwidth-heavy full file downloads.

Assessment Rubrics Summary

Criteria	Weight (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of scenario & context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand